

GaSim: A Python Class to Generate Simulated Time Signals for Gamma Spectroscopy

Z. Chen^{a,b}, D. Bykhovsky^c, X. Zheng^{a,b}, T. Trigano^{d,*}, Y. Zhu^{a,b}

^a*Shanghai Advanced Research Institute, Chinese Academy of Sciences, Shanghai, China*

^b*University of Chinese Academy of Sciences, Beijing, China*

^c*SCE, Department of Electrical Engineering, Beer-Sheva Campus, Israel*

^d*SCE, Department of Electrical Engineering, Ashdod Campus, Israel*

Abstract

The processing of nuclear pulse signals based on deep learning (DL) requires a well-labeled data set. However, the current energy spectrometers can only give users the final results, and do not allow manual labeling during the pulse signal collection process. The presented gamma_simulator (GaSim) is a Python-based gamma pulse simulator of the raw detector electrical output signal with excellent customization capabilities. It allows customization of gamma pulse signal parameters from various aspects, making it versatile and useful for a wide range of detectors. Additionally, it provides the required labels for each generated electrical pulse at specified positions, enabling the creation of datasets for DL development.

Keywords: Gamma spectroscopy, time-domain signals, simulation

Metadata

1. Motivation and significance

Gamma spectroscopy utilizes the energy spectrum derived from pulse signal energies for qualitative and quantitative analysis of radioactive sources. This technique is critical in fields such as medicine, chemical analysis, and environmental protection [1, 2, 3]. Recently, the rapid development of deep learning (DL) has shown potential for enhancing spectroscopic analyses by enabling corrections and identifications directly on energy spectra [4, 5]. However, emerging research suggests that raw time-domain signals, until now often overlooked, may provide richer information than energy spectra [6].

*Corresponding author.

Email address: thomast@sce.ac.il (T. Trigano)

© 2025. Published by Elsevier.

This manuscript version is made available under the

CC-BY-NC-ND 4.0 license

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

Code metadata description	Code metadata description	Please fill in this column
C1	Current code version	v1.1
C2	Permanent link to code/repository used for this code version	https://github.com/bykhov/gamma-simulator
C3	Permanent link to Reproducible Capsule	None
C4	Legal Code License	The 3-Clause BSD License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python
C7	Compilation requirements, operating environments & dependencies	numpy, scipy, matplotlib, urllib
C8	If available Link to developer documentation/manual	
C9	Support email for questions	chenzk@sari.ac.cn

Table 1: Code metadata (mandatory)

In this context, developing modern DL techniques that process raw time-domain signals is crucial, but the lack of annotated datasets specific to this domain hampers progress. Data acquisition is prohibitively costly and labor-intensive, often resulting in data that lacks essential labels and characteristics for effective DL training. A feasible solution is to generate simulated time-domain signal databases, where data annotation, including metrics such as arrival times, pulse durations, and energy metrics, is integrated into the simulation process.

In gamma spectroscopy, simulation techniques and supervised learning methods are commonly used to address specific issues. For instance, Monte Carlo methods have been used to simulate samples and generate energy spectra to reduce undesired physical artifacts [7]. This method has been validated and proven to be a practical tool. Moreover, simulation methods offer a wide range of adjustable parameters, allowing users to customize the simulated spectra, as demonstrated in [8]. It is worth also noting that existing simulators, such as [9], can generate pulse pile-up clusters while accounting for both paralyzable and non-paralyzable dead times.

Compared to existing simulators that generate individual pulse pile-up clus-

ters [9], our simulator offers certain improvements. With the advancement of computational capabilities, memory constraints are no longer a concern to simulate large signals instead of simulating pile-ups within a limited time frame. Our simulator generates the starting times of all pulses according to a Poisson process, which can simulate pulses and pulse clusters as one across the entire time series. This capability is important for constructing large datasets required for deep learning applications, particularly for pile-up correction. Our method preserves the flexibility to allow users to define simulation pulse parameters based on their specific requirements while adhering to physical principles. Additionally, the Python-based implementation significantly enhances the portability of the simulator, making it accessible for broader use.

In this framework, a robust simulator must not only meet the needs of DL, but also maintain physical authenticity. The presented `gamma_simulator` (GaSim) is a Python class that provides users with an interface through which they can modify the characteristics of radioactive sources and test environments, offering great flexibility. This includes, but is not limited to, the type of signal source, the energy output of a single pulse, the emission count rate of the signal source, and the ability to accurately time-label the incoming pulse signal. This flexibility can greatly contribute to the study of pulse signals, and can serve as a user guide for constructing DL datasets, as well as a signal generator for validating machine learning algorithms developed in the field. Furthermore, having been developed in Python, the simulator offers enhanced portability and facilitates collaborative improvements.

2. Software description

2.1. Software architecture

For continuous time, the output signal in gamma spectroscopy experiments can be expressed by a sample path of a shot-noise process, that is

$$y(t) = \sum_{n=0}^{\infty} E_n h_n(t - T_n) + \varepsilon(t), \quad (1)$$

where the sequence $\{T_n, n \geq 0\}$ represents the arrival times of particles and is a sample path of a homogeneous Poisson process with intensity λ , $h_n(\cdot)$ represents the shape of the electrical pulse associated with the n -th impinging particle, E_n represents its energy, and $\varepsilon(t)$ indicates the additive noise. This signal is sampled every Δt , which denotes the sampling period to produce the discrete-time signal $y[n]$. The GaSim class is organized as shown in Fig 1. Based on the parameters set by the user, the program randomly

draws the energies, arrival times, and shapes of individual electrical pulses, and organizes them so that the observed signal is a sample path of a shot-noise process, commonly used in the field [10]. The energies are drawn based on real energy spectra freely accessible online.

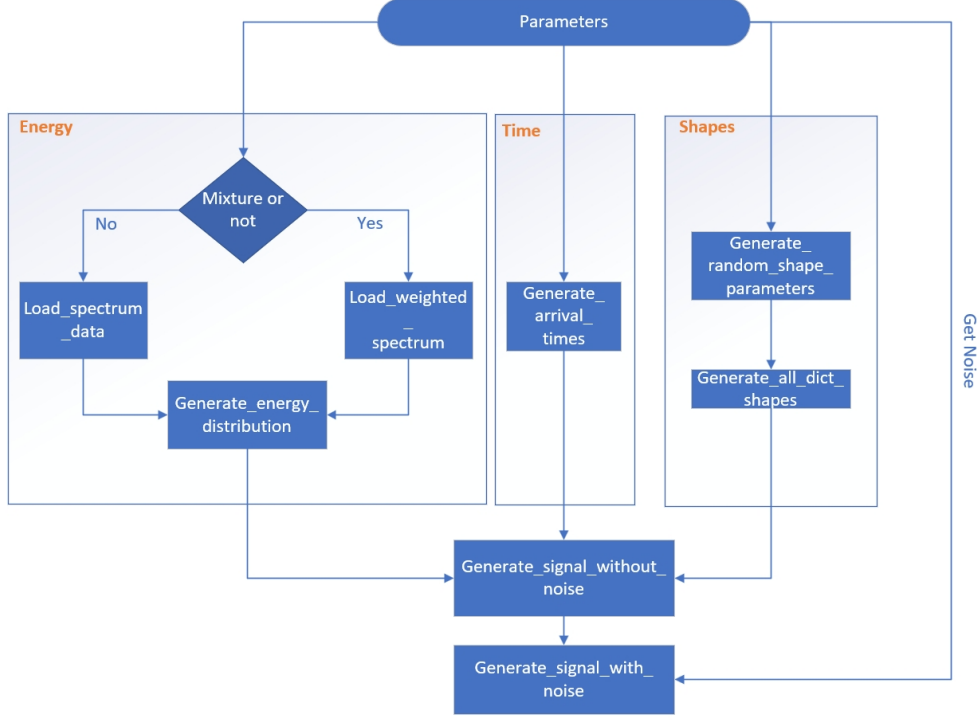


Figure 1: Flow Chart.

Table 2 enumerates the main function names present in the entire code along with their respective functionalities. A more detailed description of these functions can be found in Subsection 2.2.

2.2. Software functionalities

2.2.1. Pulse arrival time

The arrival time series is represented by the events T_n of the Poisson process in Eq. 1. The arrival time series conforming to a specific Poisson distribution is the most plausible model of the physical principles and count rates of a real signal, and is a crucial factor in determining whether the analog signal resembles the real signal. The mathematical expression of the Poisson process is as follows:

$$\Pr(N(t) = n) = \frac{(\lambda t)^n}{n!} \exp(-\lambda t), \quad (2)$$

Function name	Inputs	Output	Short description
generate_energy_distribution	seed, n_events, hist_energy, hist_counts	energies	Generates n_event pulse energy sequences conforming to the probability density distribution of the target energy spectrum
generate_arrival_times	seed, lambda_value	times	Generates a Poisson process event arrival time series with a specified count rate (lambda_value)
generate_all_dict_shapes	dic_type, dict_shape_params, dict_size	shapes	Produces a shape dictionary of the specified shapes generated by the specified parameters as a selection library of energy shapes
generate_signal_without_noise	times, energies, shapes.len, shapes	signal without noise	Generates analog data without noise
generate_signal_with_noise	signal without noise, noise_unit, noise	signal	The noiseless signal is superimposed on the specified unit and size of noise to obtain a real analog signal

Table 2: Functions definition.

where $N(t)$ denotes the number of events until time t and λ is the average rate of arrivals per time unit. It is known that the time interval between arrivals in a Poisson process follows an exponential distribution through a simple mathematical derivation:

$$\Pr(T_{n+1} - T_n < t) = 1 - \Pr(N(t) = 0) = 1 - \exp(-\lambda t), \quad (3)$$

so that the inter-arrival times are independent and identically distributed exponential random variables. Well-developed standard methods can be used to generate sequences that fit a particular exponential distribution [11]. When

the function receives the input `lambda_value`, it can output a time series called `times` with a specified counting rate.

2.2.2. Pulse energy sequence

Different radioisotopes have different and specific energy spectra, and the energy sequences conforming to these spectra are unique identifiers that can distinguish between different radioisotopes. Once the arrival times have been generated, the energies associated with each particle can be obtained. In this paper, we used energy spectra stemming from the Gamma-Spectrum-Database [12]; however, that input can also be user-defined, e.g. obtained from simulators such as MCNP [13] or PHITS [14]. The time series of the generated energies, denoted by E_n in Eq. (1), are drawn based on the provided energy spectra using an accept-reject procedure [11].

2.2.3. Pulse shape dictionary

Once the energy series and time series have been determined, the shapes of the pulses represented by $h_n(\cdot)$ in Eq. 1 are generated in this function. The root cause of pulse stacking is the superposition of the electrical pulses' shapes. Therefore, a library of simulated shapes that match real pulse shapes and lengths can be very helpful for achieving robust stacked pulse correction methods on real signals in the future. For shape estimation, dictionary-learning algorithms such as [15] can be used to create a dictionary of shapes for signal characterization.

Although other models can also be applied [16, 17], the most widely used physical models for this type of fast rise and slow fall are the double exponential model and the gamma model. The double exponential model is given by [18]

$$h(t) = a \left(\exp \left(-\frac{t}{\tau_2} \right) - \exp \left(-\frac{t}{\tau_1} \right) \right), \quad (4)$$

where a is the normalization factor and $\tau_1 \ll \tau_2$ are characteristic decay times. The gamma-function model can be expressed as

$$h(t) = at^\alpha e^{-\beta t}, \quad (5)$$

where a is a normalization factor, α is the shape parameter, and β is the scale parameter.

Model parameters may be estimated directly using different fitting techniques [19], as these signals are typically low-activity signals. However, in practice, the detector shape has inherent randomness due to physical inhomogeneities, temperature fluctuations, and other factors related to detector physics. Therefore, a single precise parameter set cannot perfectly fit

the pulse shape, leading us to generate our parameter set randomly from a Gaussian distribution with reasonable expectations and variances [20].

2.2.4. Additive noise

At this stage, an ideal noiseless signal can be generated by the function `generate_signal_without_noise`. However, the simulation is not yet complete, because any practical electrical measurement is affected by additive noise, which can be modeled as $\varepsilon(t)$ in Eq. 1. This noise can originate from various sources, including the thermal motion of electrons in an electrical conductor (thermal noise), electromagnetically induced environmental noise, quantization noise, and others. The addition of realistic noise signals will make analog signals more realistic and are crucial for investigating noise cancellation methods. When all these noises are combined, they are typically modeled as additive white Gaussian noise (AWGN), according to the central limit theorem.

The simulator proposes two ways to quantify the amount of noise:

- Define the noise variance, σ_n ;
- Define the signal-to-noise ratio (SNR), which measures the ratio between the power of the signal and the power of the noise. Note that the power of the AWGN is linearly proportional to the AWGN variance.

At each sampling point, the function `generate_signal_with_noise` generates a user-defined noise value, which is then added to the original pulse signal. This process simulates the effect of noise on the pulse and reflects the final output signal under the influence of noise.

3. Illustrative examples

In this section, we provide a simple example (the simulation of a ^{137}Cs energy spectrum and time signal) to demonstrate the software’s capabilities. Furthermore, we also compare the simulator results with energy spectra from real-life experiments to validate the accuracy of the simulator. Energy spectra collected from experiments at the Shanghai Synchrotron Radiation Facility (SSRF) were employed for this purpose.

3.1. ^{137}Cs detector output generation

In this section we present a case study (^{137}Cs) to illustrate the simulator’s output. First, we investigate whether the obtained energy spectra at very low activity look similar to those in the dataset. To do so, we simulate a

signal stemming from a ^{137}Cs source with an activity of 1000 counts per second (cps), sampled at 1 MHz, and compare the resulting histogram obtained from the time signal by thresholding the electrical pulses and computing the energies. Results are shown in Fig. 2, which demonstrate close similarity between the two spectra.. The code used to generate the associated time signals is displayed in Listing 1.

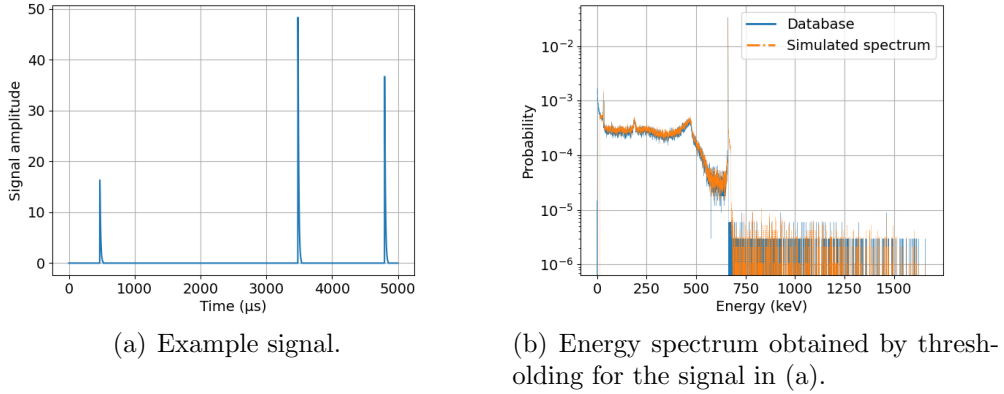


Figure 2: Simulation of low-counting rate ^{137}Cs from [12]: (a) signal, (b) energy distribution, comparing simulated spectrum with spectrum from database.

Additionally, we assessed the simulator’s performance under a relatively high count rate for ^{137}Cs . A signal of 100 seconds with a count rate of 40000 cps was generated, with all other parameters remaining the same. Figure 3 presents a part of the generated time signal, while Fig 3(b) displays the energy spectrum of the pulse sequence derived from the obtained time signal. We observe that the accumulated peak values predominantly cluster around integer multiples of 662 keV, aligning with the theory of pulse accumulation and observed pile-ups in practice.

3.2. Performance Validation

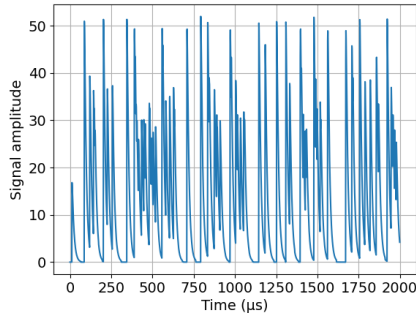
To validate the effectiveness of the simulator, energy spectra derived from the simulated time signals based on spectra from [12] are compared with energy spectra obtained from experiments performed at the Shanghai Synchrotron Radiation Facility (SSRF) using ^{137}Cs and ^{60}Co sources. The HPGe detector used in the study is the GEM-70200-P located 0.1m from the source, while the equipment responsible for converting electrical signals into digital signals is the CAEN V1725S model. We compare the energy spectrum obtained from the 1000 s of generated time signal with a counting rate of 1178 cps with that of real data, as shown in Fig. 4. The results reveal good adequacy between

Listing 1: ^{137}Cs signal simulation.

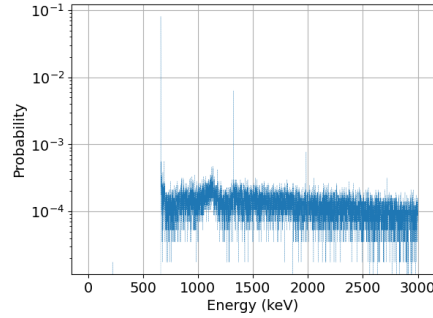
```

1 from gamma_simulator import gamma_simulator
2 simulator = gamma_simulator(verbose=True,
3                             verbose_plots={'shapes': True,
4                                           'signal': True},
5                             source={'name': 'Cs-137', 'weights': 1},
6                             signal_len=1000, # sec
7                             fs=1e6, # Hz
8                             lambda_value=1000, # counts-per-second
9                             dict_type='gamma',
10                            dict_shape_params={'mean1': 0.1,
11                                              'std1': 0.001,
12                                              'mean2': 1e5,
13                                              'std2': 1e3},
14                            noise_unit='std',
15                            noise=1e-3,
16                            dict_size=10,
17                            seed=42)
18 s = simulator.generate_signal()

```



(a) Time signal obtained for counting rate 40000 cps.



(b) Energy spectrum obtained by thresholding for the signal in (a).

Figure 3: Simulation of high-counting rate ^{137}Cs : (a) signal, (b) obtained histogram. The differences observed here compared to Figure 2 are due to the significant difference in counting rates.

the simulator-generated spectrum and the experimental spectrum. Discrepancies below 200 keV stem from different detection thresholds used in our simulator and the actual detector at the SSRF. At 600 keV, the experimen-

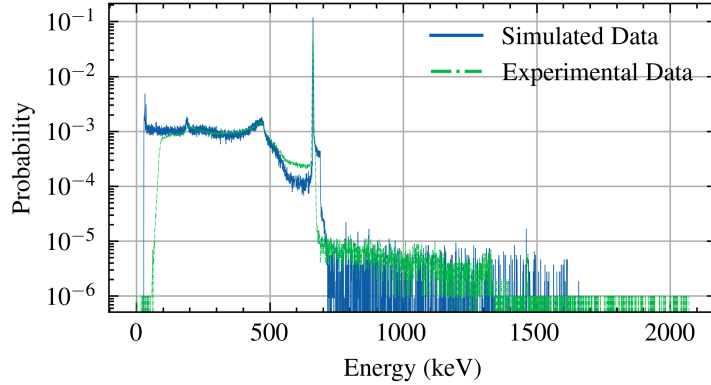


Figure 4: Comparison between a ^{137}Cs energy spectrum derived from the time signal obtained by GaSim (blue) and an energy spectrum obtained from experimental measurements at SSRF (green), for a counting rate of 1178 cps.

tal data represents the energy spectrum obtained without any algorithmic correction applied to the digital signals for the Compton effect attenuation, leading to a higher Compton edge in the real energy spectrum. More generally, differences between the detector's type, size and geometry, as well as the use or not of Compton correction algorithms, can likely explain the observed discrepancies in the Compton continuum between simulation and experimental results. The height difference in the full-energy peak can be attributed to the influence of the detector size. However, this does not affect the position of the full-energy peak, which is our primary focus. Overall, the simulated energy spectrum aligns well with that derived from actual data, despite its having been derived from a different sensor.

Another example is related to ^{60}Co . We conducted a comparison between the simulated energy spectrum of the ^{60}Co radioisotope and the corresponding experimental result. The simulation duration was set to 1800 s, and the count rate was 1267 cps. The code is displayed in Listing 2. Results of the energy spectrum, shown in Fig 5, revealed a clear alignment of the peaks of interest. Discrepancies at high energies between experimental and simulated results stem from differences in experimental setups. Indeed, the flatness observed above 1400 keV in the experimental spectrum arises from the proximity of the ^{60}Co source to the detector. This phenomenon is further evidenced by the distinct ^{60}Co sum peak at 2505 keV (1332 + 1173 keV), highlighted within the red box.

Listing 2: ^{60}Co signal generation.

```

1 from gamma_simulator import gamma_simulator
2 simulator = gamma_simulator(verbose=True,
3                               source={'name': 'Co-60', 'weights': 1},
4                               signal_len=1800, # sec
5                               fs=1e6, # Hz
6                               lambda_value=1267, # counts-per-second
7                               dict_type='gamma',
8                               dict_shape_params={'mean1': 0.1,
9                                                  'std1': 0.001,
10                                                  'mean2': 1e5,
11                                                  'std2': 1e3},
12                               noise_unit='std',
13                               noise=1e-3,
14                               dict_size=10,
15                               seed=42)
16 s = simulator.generate_signal()

```

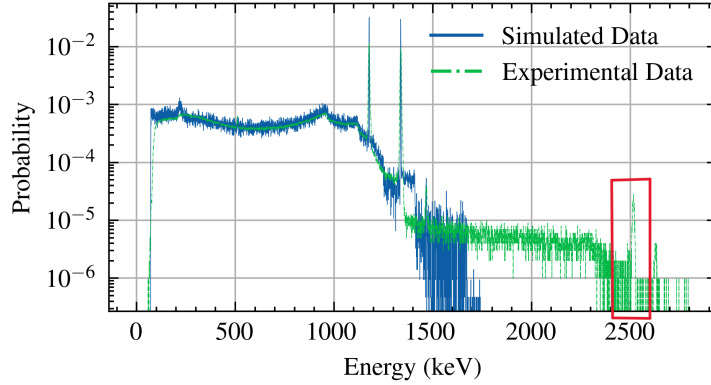


Figure 5: Comparison between a ^{60}Co energy spectrum derived from the time signal obtained by GaSim (blue) and an energy spectrum obtained from experimental measurements at SSRF (green), for a counting rate of 1267 cps.

4. Impact

GaSim is capable of tailoring pulse signals to meet user specifications. The recent successes of DL in various domains show the significant potential of its application to nuclear pulse signal processing. Regardless of the signal processing methods used, the ultimate goal is to obtain an accurate energy spectrum. However, existing instruments provide final results without allow-

ing any modifications or customization, resembling a sealed black box. The current limitations in obtaining a dataset with specific annotations hinder gamma pulse analysis, deterring many who wish to employ DL methods in this area.

GaSim displays two notable advantages:

- Highly customizable datasets enhance the feasibility of DL in the domain of nuclear pulse signal analysis.
- The arduous and expensive process of collecting gamma pulse signals is circumvented.

In conclusion, GaSim Pulse Simulator can be a valuable tool in future nuclear pulse processing. With the integration of DL methods, the focus on nuclear pulse signal processing may shift from improving instruments and existing analytic algorithms to optimizing the impact of DL, with predictable advancements in its development.

5. Conclusions

We have completed the development of a simulator that customizes gamma pulse signals based on various factors, such as the type of radioactive source, count rate, sampling rate, noise, etc. This allows developers to obtain a desired dataset. Additionally, through open source availability, future researchers can customize arbitrary labels for the signals, which is beneficial for research work.

Acknowledgment

This research was supported by the National Natural Science Foundation of China under grant numbers 12373113 and 12475196.

References

- [1] R. Schafer and P. Schmidt. *Methods in Physical Chemistry*. Wiley, 2012. <https://onlinelibrary.wiley.com/doi/book/10.1002/9783527636839>.
- [2] Christian Baker, Daniel Sarno, Robert J. Eckersley, and Bajram Zeqiri. Pulse pileup correction of signals from a pyroelectric sensor for phase-insensitive ultrasound computed tomography. *IEEE Transactions on Instrumentation and Measurement*, 68(10):3920–3931, 2019. <https://doi.org/10.1109/TIM.2018.2884579>.

- [3] Mojca Miklavc, Bastian Löher, Deniz Savran, Roman Novak, Simon Širca, and Matjaž Vencelj. Pile-up correction techniques for real-time dosimetry in photon radiotherapy. In *2012 IEEE Nuclear Science Symposium and Medical Imaging Conference Record (NSS/MIC)*, pages 3880–3882, 2012. <https://doi.org/10.1109/NSSMIC.2012.6551889>.
- [4] Ruihao Luo, Juergen Popp, and Thomas Bocklitz. Deep learning for raman spectroscopy: A review. *Analytica*, 3(3):287–301, 2022. <https://doi.org/10.3390/analytica3030020>.
- [5] Bulent Ayhan and Chiman Kwan. New results on radioactive mixture identification and relative count contribution estimation. *Sensors*, 21(12), 2021. <https://doi.org/10.3390/s21124155>.
- [6] Tom Trigano and Dima Bykhovsky. Deep learning based method for activity estimation from short-duration gamma spectroscopy recordings. *IEEE Transactions on Instrumentation and Measurement*, 2024. accepted for publication.
- [7] Michael Iro, Dieter Ingerle, Sven Hampel, Ursula Fittschen, Vishal Dhamgaye, Oliver Fox, and Christina Streli. voxtrace: A voxel-based monte-carlo ray-tracing code for the simulation of x-ray fluorescence spectra. *SoftwareX*, 23:101481, 2023. <https://doi.org/10.1016/j.softx.2023.101481>.
- [8] Danny Petschke and Torsten E.M. Staab. Ddrs4pals: A software for the acquisition and simulation of lifetime spectra using the drs4 evaluation board. *SoftwareX*, 10:100261, 2019. <https://doi.org/10.1016/j.softx.2019.100261>.
- [9] Ali Asghar Mowlavi and Mohammad Hadi Hadizadeh Yazdi. Monte carlo simulation of pulse pile-up effect in gamma spectrum of a pgnaa system. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 660(1):104–107, 2011. <https://doi.org/10.1016/j.nima.2011.09.022>.
- [10] G. F. Knoll. *Radiation Detection and Measurement*. Wiley, Hoboken, N.J, 4 edition edition, August 2010.
- [11] G. Casella and C. P. Robert. *Monte-Carlo Statistical Methods*. Springer, 2nd edition edition, 2004.

- [12] Gamma-spectrum-database. <https://github.com/OpenGammaProject/Gamma-Spectrum-Database>.
- [13] Joel Aaron Kulesza, Terry R. Adams, Jerawan Chudoung Armstrong, Simon R. Bolding, Forrest Brooks Brown, Jeffrey S. Bull, Timothy Patrick Burke, Alexander Rich Clark, Robert Arthur Forster, III, Jesse Frank Giron, Tristan Sumner Grieve, Colin James Josey, Roger Lee Martz, Gregg Walter McKinney, Eric J. Pearson, Michael Evan Rising, Clell Jeffrey Solomon, Jr., Sriram Swaminarayan, Travis John Trahan, Stephen Christian Wilson, and Anthony J. Zukaitis. MCNP® Code Version 6.3.0 Theory & User Manual. Technical Report LA-UR-22-30006, Rev. 1, Los Alamos National Laboratory, Los Alamos, NM, USA, September 2022.
- [14] Tatsuhiko Sato, Yosuke Iwamoto, Shintaro Hashimoto, Tatsuhiko Ogawa, Takuya Furuta, Shin-Ichiro Abe, Takeshi Kai, Yusuke Matsuya, Norihiro Matsuda, Yuho Hirata, Takuya Sekikawa, Lan Yao, Pi-En Tsai, Hunter N. Ratliff, Hiroshi Iwase, Yasuhito Sakaki, Kenta Sugihara, Nobuhiro Shigyo, Lembit Sihver, and Koji Niita. Recent improvements of the particle and heavy ion transport code system – PHITS version 3.33. *Journal of Nuclear Science and Technology*, 61(1):127–135, 2024.
- [15] Michal Aharon, Michael Elad, and Alfred Bruckstein. K-svd: An algorithm for designing overcomplete dictionaries for sparse representation. *IEEE Transactions on Signal Processing*, 54(11):4311–4322, 2006. <https://doi.org/10.1109/TSP.2006.881199>.
- [16] Xi Wang, Qingguo Xie, Yuanbao Chen, Ming Niu, and Peng Xiao. Advantages of digitally sampling scintillation pulses in pileup processing in pet. *IEEE Transactions on Nuclear Science*, 59(3):498–506, 2012. <https://doi.org/10.1109/TNS.2012.2183646>.
- [17] Minju Lee, Daehee Lee, Eunbie Ko, Kyeongjin Park, Junhyuk Kim, Kilyoung Ko, Manish Sharma, and Gyuseong Cho. Pulse pileup correction method for gamma-ray spectroscopy in high radiation fields. *Nuclear Engineering and Technology*, 52(5):1029–1035, 2020. <https://doi.org/10.1016/j.net.2019.12.003>.
- [18] Glenn F. Knoll. *Radiation detection and measurement*. John Wiley & Sons, 4th edition, 2010. <https://api.semanticscholar.org/CorpusID:16661089>.

- [19] Yuewu Shi, Wei Wang, Zhizhen Zhu, and Xin Nie. The parameter estimation of double exponential pulse based on a least infinity norm fitting method. In *Journal of Physics: Conference Series*, volume 2083, page 042002. IOP Publishing, 2021. <https://doi.org/10.1088/1742-6596/2083/4/042002>.
- [20] Ortec. *Gamma-Ray Spectroscopy Using NaI(Tl)*. <https://www.ortec-online.com.cn/-/media/ametektortec/third-edition-experiments/3-gamma-ray-spectroscopy-using-nai-tl.pdf?la=zh-cn&revision=001dbc1d-9559-49c0-b57d-5567e28d1b96&hash=0D4E147F8F47A4485C42D55776D9E075>.